

RAGstar — Entity Relationship Diagram

사용자별 차등 서비스 제공을 위한 데이터 모델 (v0.5)

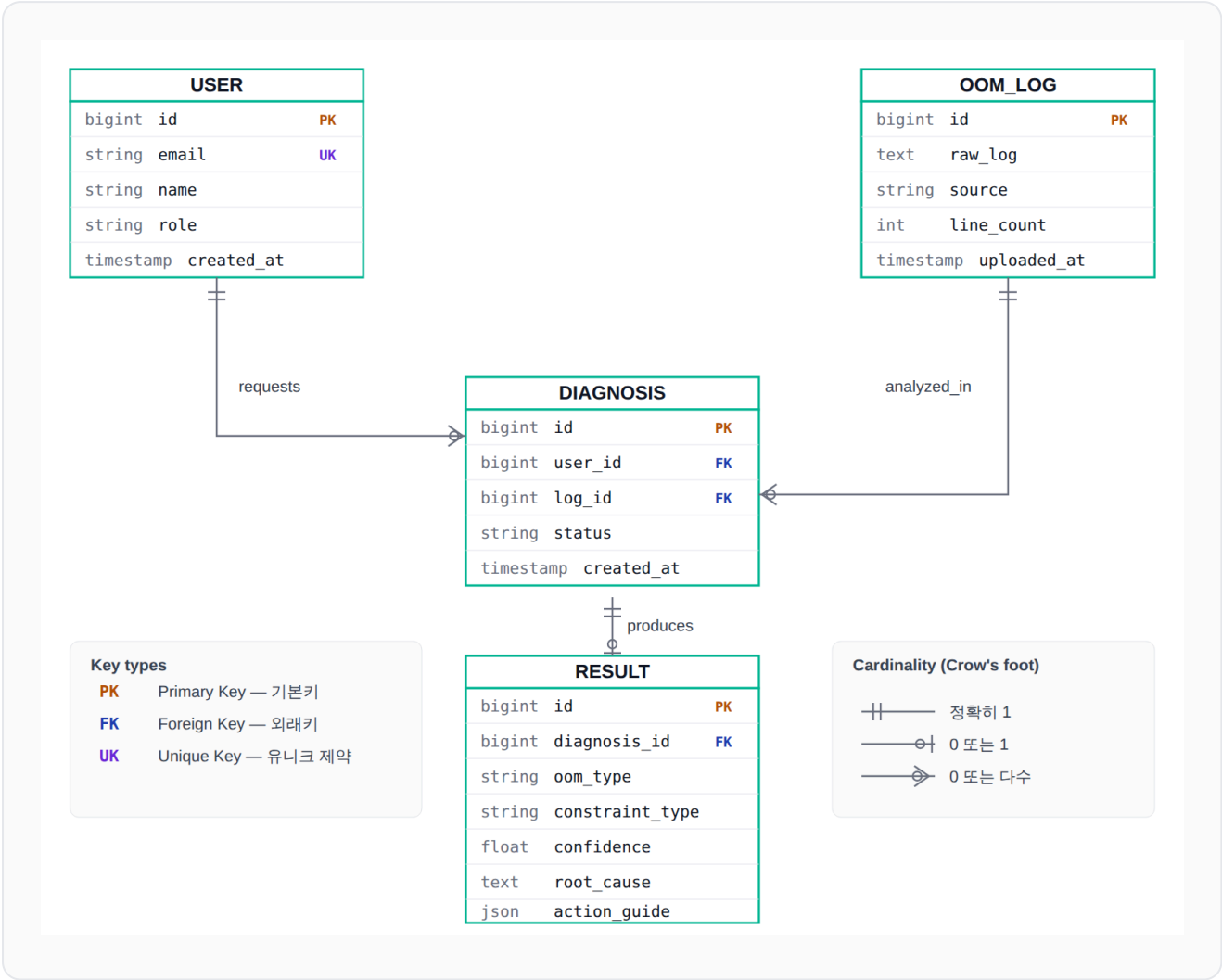


Figure 1. RAGstar 핵심 엔티티 관계도

1. 엔티티 구성

USER

시스템을 사용하는 운영자/관리자 계정. 모든 데이터의 소유 주체이자 권한 분리의 기준이 되는 최상위 엔티티. 사용자별 차등 서비스의 출발점이 된다.

id	bigint	PK	사용자 고유 식별자 (auto-increment)
email	string	UK	로그인 아이디로 사용되는 이메일 (중복 불가)
name	string		화면에 표시될 사용자 이름
role	string		권한 레벨 (admin / operator / viewer)
created_at	timestamp		계정 생성 시각

OOM_LOG

사용자가 입력한 dmesg 원본 로그를 보관하는 엔티티. raw_log는 수십 KB 이상의 큰 텍스트 필드라 자주 조회되는 진단 메타데이터(user_id, status 등)와 분리하여 저장함으로써 인덱스 효율과 조회 성능을 확보한다. 또한 향후 동일 로그를 재참조하거나 외부 시스템에서 로그만 검색해야 하는 경우 독립적인 접근이 가능해진다.

id	bigint	PK	로그 고유 식별자
raw_log	text		dmesg 원본 텍스트 전체 (파싱 전 상태로 보존)
source	string		입력 경로 (paste / file_upload / api)
line_count	int		로그 라인 수 (UI 표시 및 통계용)
uploaded_at	timestamp		로그 업로드 시각

DIAGNOSIS

진단 요청 1건을 표현하는 중심 트랜잭션 테이블. USER, OOM_LOG, RESULT를 하나로 묶어 "누가, 어떤 로그를, 언제 분석해서, 어떤 결과를 얻었는지"를 추적한다. 사용자별 이력 조회의 핵심 진입점.

id	bigint	PK	진단 고유 식별자
user_id	bigint	FK	진단을 요청한 사용자 (USER.id 참조)
log_id	bigint	FK	분석 대상 로그 (OOM_LOG.id 참조)
status	string		처리 상태 (pending / running / success / failed)
created_at	timestamp		진단 요청 시각 (이력 정렬 기준)

RESULT

RAG 파이프라인이 산출한 진단 결과. UI의 "진단 결과" 카드에 표시되는 모든 데이터(OOM 유형, 제약 조건, 신뢰도, 근본 원인, 조치 가이드)를 담는다. 진단이 실패한 경우 RESULT가 생성되지 않을 수 있어 DIAGNOSIS와는 0..1 관계로 연결된다.

id	bigint	PK	결과 고유 식별자
diagnosis_id	bigint	FK UK	결과가 속한 진단 (DIAGNOSIS.id 참조, 1:1 강제용 UK)
oom_type	string		분류된 OOM 유형 (예: CGROUP_OOM, GLOBAL_OOM)
constraint_type	string		제약 조건 (CONSTRAINT_MEMCG, CONSTRAINT_NONE 등)
confidence	float		분류 신뢰도 (0.0 ~ 1.0)
root_cause	text		LLM이 생성한 근본 원인 설명문
action_guide	json		조치 가이드 항목 리스트 (단계별 권고사항)

2. 관계 핵심

USER — DIAGNOSIS · 1:N (requests)

USER ||--o{ DIAGNOSIS

한 사용자는 여러 건의 진단을 요청할 수 있고, 각 진단은 반드시 한 사용자에게만 귀속된다. `DIAGNOSIS.user_id`를 통해 사용자별 진단 이력을 시간순으로 조회할 수 있으며, 이는 "내 진단 이력" 화면의 기반이 된다. 이 관계가 사용자별 차등 서비스의 핵심으로, 모든 진단 데이터는 user_id 필터를 통해 본인 데이터만 노출되도록 격리된다.

OOM_LOG — DIAGNOSIS · 1:N (analyzed_in)

OOM_LOG ||--o{ DIAGNOSIS

진단 1건은 정확히 하나의 로그를 분석하지만, 한 로그는 향후 여러 진단의 입력으로 재참조될 수 있도록 1:N으로 모델링하였다. raw_log는 큰 텍스트 필드이므로 진단 메타데이터와 분리해 저장하면 DIAGNOSIS 테이블의 행 크기가 작아져 인덱스 효율과 이력 조회 성능이 향상되며, 향후 로그 자체를 검색하거나 외부 시스템에서 참조하는 경우에도 독립적으로 접근할 수 있다.

DIAGNOSIS — RESULT · 1:0..1 (produces)

DIAGNOSIS ||--o| RESULT

진단이 성공적으로 완료되면 정확히 하나의 결과가 생성되며, 진단이 진행 중이거나(**running**) 실패한(**failed**) 경우에는 RESULT가 존재하지 않을 수 있다. 외래키는 **RESULT.diagnosis_id** 에 두어 "진단이 먼저 만들어지고 그 결과가 나중에 첨부된다"는 실제 흐름과 일치시켰으며, UNIQUE 제약으로 한 진단당 결과가 둘 이상 생기지 않도록 강제한다.

3. 사용자별 차등 서비스 시나리오

시나리오 A — 운영자 김씨가 OOM 로그를 진단

1. 로그인 시 **USER.id = 42** 식별, JWT에 담아 모든 API 요청에 첨부
2. 로그를 입력하면 **OOM_LOG** 신규 레코드 생성, 동시에 **DIAGNOSIS** 에 **user_id=42, log_id=새로 만든 로그 id, status='running'** 으로 진단 레코드 생성
3. RAG 파이프라인이 성공하면 **RESULT** 를 신규 생성하고 **diagnosis_id** 로 진단에 연결, **DIAGNOSIS.status** 를 **'success'** 로 업데이트
4. 파이프라인이 실패하면 RESULT 없이 **DIAGNOSIS.status='failed'** 만 기록 (실패 진단도 이력에 남음)

시나리오 B — "내 진단 이력" 조회

- **SELECT * FROM DIAGNOSIS WHERE user_id = 42 ORDER BY created_at DESC** — 본인 진단만 노출되어 다른 사용자 데이터는 격리된다.
- OOM_LOG와 RESULT는 LEFT JOIN으로 가져와 한 화면에 통합 표시 (성공 건은 oom_type 표기, 실패 건은 status 뱃지로 구분).
- raw_log가 큰 텍스트라 이력 목록 화면에서는 OOM_LOG.line_count만 노출하고, 상세 화면 진입 시에만 raw_log를 로드하여 초기 렌더링 속도를 확보한다.